

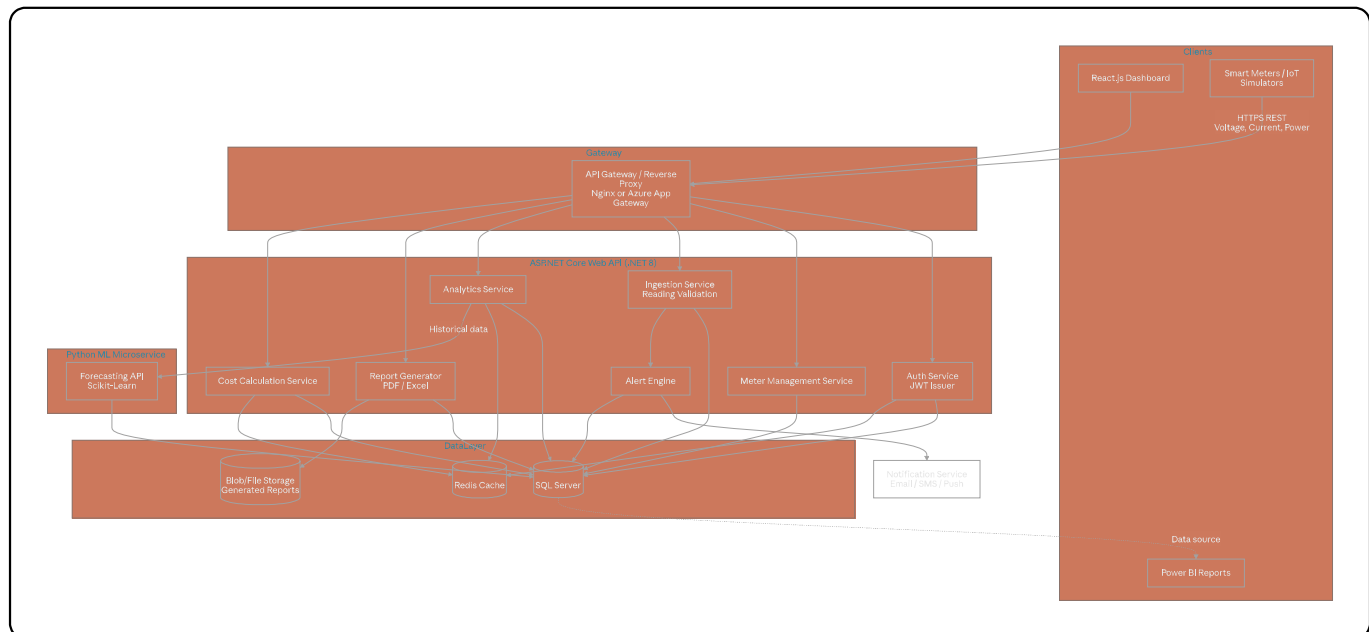
Smart Energy Monitoring & Analytics System — System Design Document

1. Overview

A web-based platform for monitoring, analyzing, and optimizing electricity consumption across industrial plants, offices, warehouses, or residential communities. Smart meters (real or simulated IoT) feed readings into a backend API, which stores them in SQL Server, surfaces them through dashboards, generates alerts on abnormal usage, calculates costs, and forecasts future consumption.

Stack: ASP.NET Core Web API (.NET 8) · SQL Server · React.js · JWT Auth · Redis (caching) · Python/Scikit-Learn (ML microservice) · Power BI (reporting) · Docker / Azure or AWS (deployment)

2. High-Level Architecture

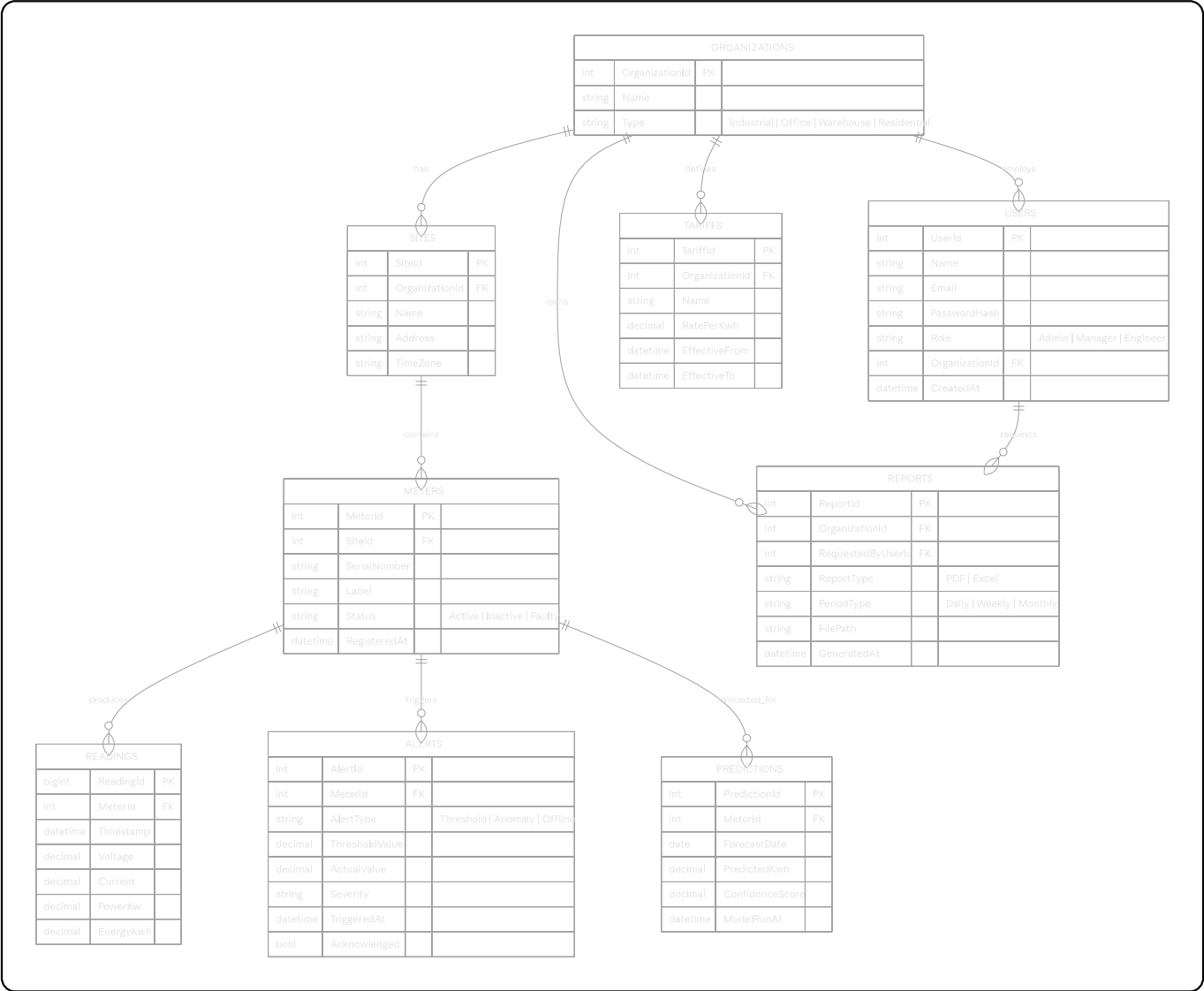


Key design notes:

- The API Gateway is the single entry point for both meter ingestion traffic and dashboard/UI traffic, enabling centralized auth, rate-limiting, and routing.
- Redis caches JWT validation results, tariff rates, and frequently-queried dashboard aggregates (e.g., "today's consumption per plant") to reduce SQL Server load.
- The ML microservice is a separate Python process (FastAPI/Flask + Scikit-Learn) called over REST from the Analytics Service — keeping ML retraining/deployment independent of the .NET release cycle.

- Alert Engine runs both synchronously (on ingestion, for instant threshold breaches) and via a scheduled background job (for rolling-window anomalies like "20% above weekly average").

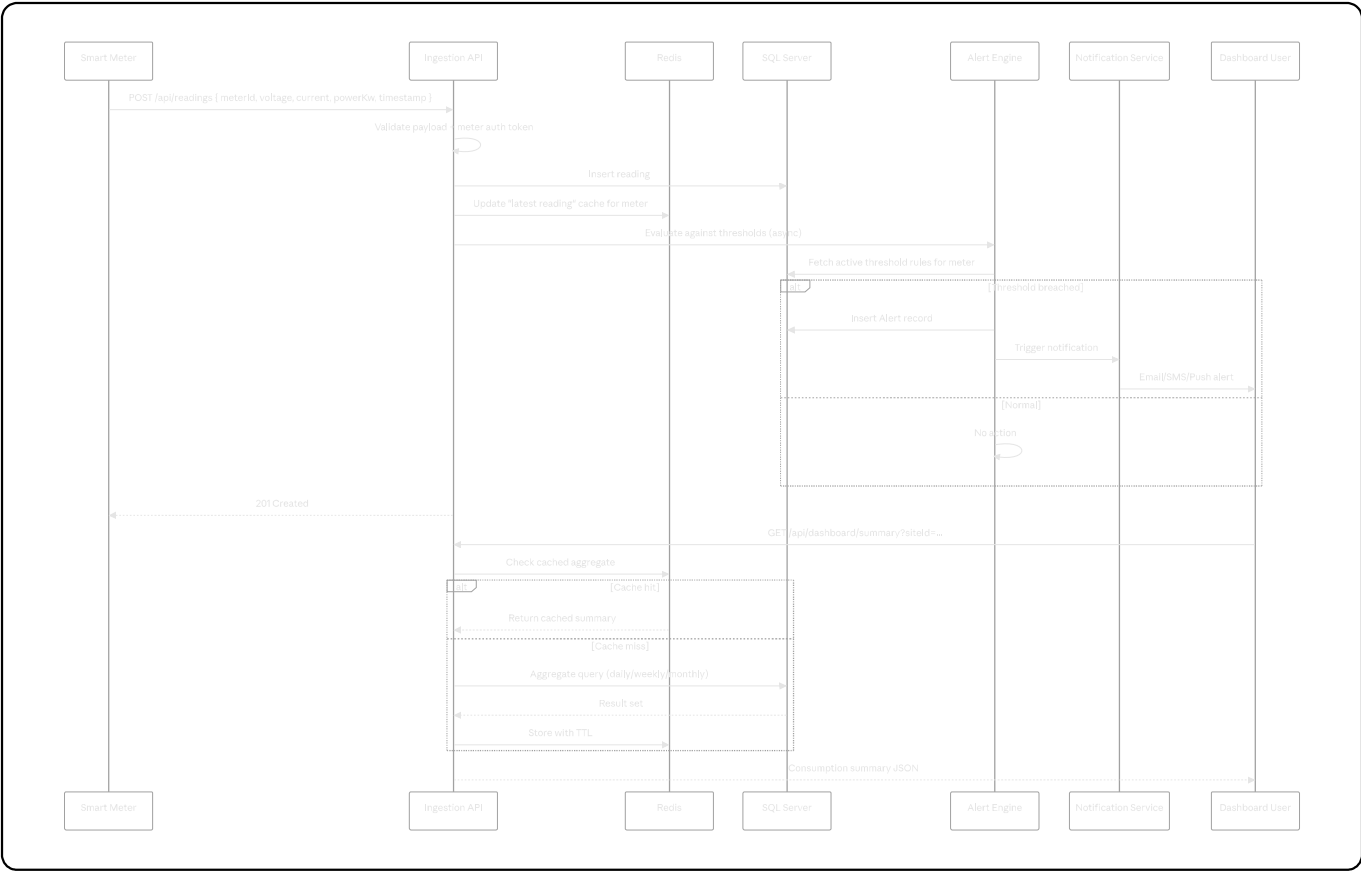
3. Database Schema (ER Diagram)



Indexing notes:

- READINGS(MeterId, Timestamp)** composite index (or a clustered columnstore index) is essential — this table grows fastest and dashboard/analytics queries always filter by meter + time range.
- Consider partitioning **READINGS** by month once data volume grows, and rolling up old raw readings into hourly/daily aggregate tables for long-term analytics.

4. Core Workflow — Meter Reading Ingestion & Alerting



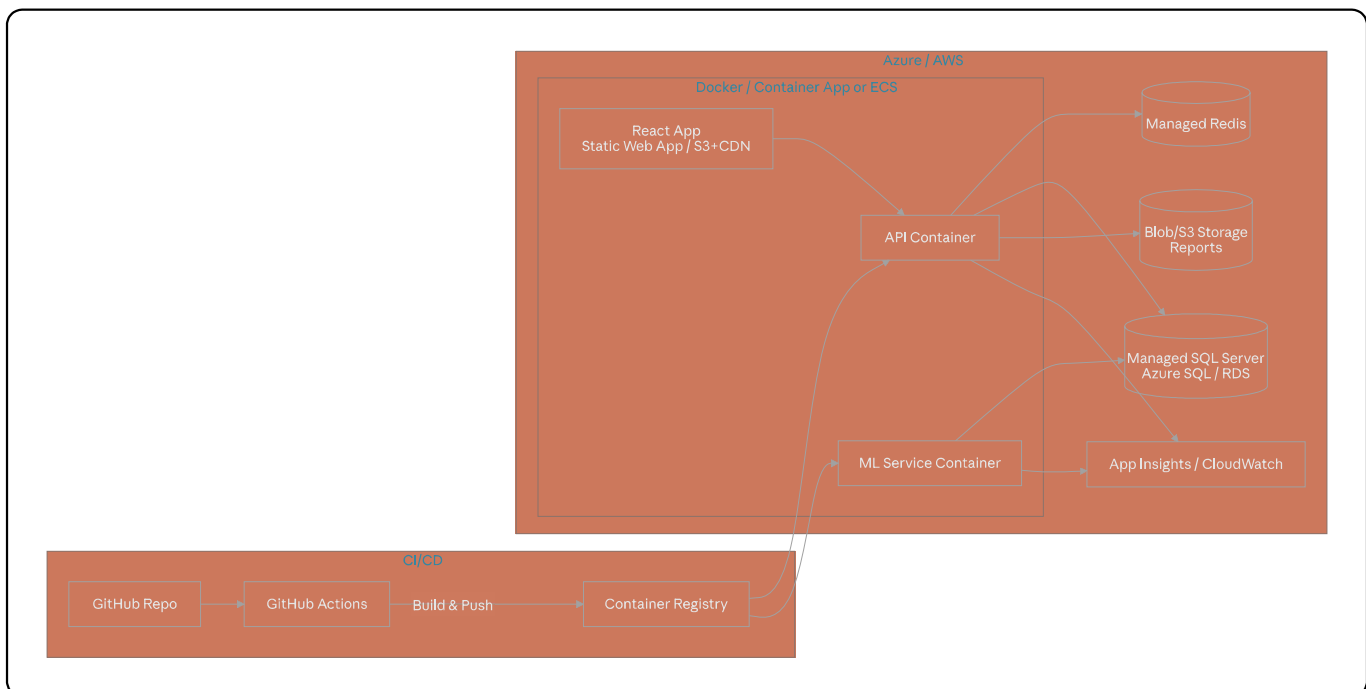
5. API Surface (Summary)

Module	Endpoint	Method	Role(s)
Auth	<code>/api/auth/login</code>	POST	Public
Auth	<code>/api/auth/refresh</code>	POST	Authenticated
Meters	<code>/api/meters</code>	GET/POST	Admin, Manager
Meters	<code>/api/meters/{id}</code>	GET/PUT/DELETE	Admin
Readings	<code>/api/readings</code>	POST	Meter device token
Readings	<code>/api/readings/{meterId}</code>	GET	Manager, Engineer
Dashboard	<code>/api/dashboard/summary</code>	GET	All roles
Alerts	<code>/api/alerts</code>	GET	Manager, Engineer
Alerts	<code>/api/alerts/{id}/acknowledge</code>	POST	Manager, Engineer
Cost	<code>/api/cost/calculate</code>	GET	Manager

Module	Endpoint	Method	Role(s)
Analytics	<code>/api/analytics/peak-usage</code>	GET	Manager, Engineer
Analytics	<code>/api/analytics/top-consumers</code>	GET	Manager, Engineer
Predictions	<code>/api/predictions/{meterId}</code>	GET	Manager
Reports	<code>/api/reports/generate</code>	POST	Manager, Admin
Reports	<code>/api/reports/{id}/download</code>	GET	Manager, Admin

Full request/response schemas will live in Swagger/OpenAPI, auto-generated from the .NET controllers (Swashbuckle).

6. Deployment Architecture



Notes:

- Each component (.NET API, ML service, React app) ships as its own container/image, orchestrated via `docker-compose` locally and Azure Container Apps / AWS ECS in production.
- SQL Server and Redis use managed cloud services rather than self-hosted containers in production, for backup/failover and patching.
- App Insights (Azure) or CloudWatch (AWS) handle centralized logging, metrics, and alerting on the infrastructure itself (separate from the application's own energy-alerting feature).

7. Role-Based Access Summary

Role	Permissions
Admin	Full access: manage users, organizations, sites, meters, tariffs; view everything
Manager	View dashboards, analytics, costs, predictions; acknowledge alerts; generate reports
Engineer	View dashboards, raw readings, alerts; acknowledge alerts; no cost/tariff visibility

8. Suggested Build Order (Roadmap)

1. **Foundation:** SQL Server schema + EF Core models/migrations; Auth (JWT + roles)
 2. **Core API:** Meter CRUD, Reading ingestion endpoint, basic validation
 3. **Dashboard data:** Aggregation queries + Redis caching layer
 4. **Alerting:** Threshold rules + Alert Engine + notification stub (log/email)
 5. **React frontend:** Login, meter list, live dashboard charts
 6. **Cost & Analytics:** Tariff-based cost calc, peak-usage/top-consumer queries
 7. **Reports:** PDF/Excel generation
 8. **ML service:** Python forecasting microservice + integration endpoint
 9. **Power BI:** Connect to SQL Server / API for executive reporting
 10. **Dockerize & deploy:** docker-compose → cloud deployment + CI/CD
-

Next step: scaffold the .NET 8 Web API solution (Step 1–2 above) with the database schema as EF Core migrations and a working `/api/readings` + `/api/auth/login` endpoint.